

// Jivops

MongoDB

Guía completa — 20 lecciones

Fundamentos y modelado · Índices y rendimiento · Replicación y sharding ·
Seguridad, backup y producción

Contenido

Módulo 1: Fundamentos y modelado de datos

- Lección 01 — Qué es MongoDB y su modelo de documentos
- Lección 02 — Modelado — incrustar o referenciar
- Lección 03 — Validación de esquema con \$jsonSchema
- Lección 04 — Operadores de actualización y atomicidad por documento
- Lección 05 — Transacciones multi-documento

Módulo 2: Índices y rendimiento

- Lección 06 — Índices — compuestos (regla ESR), multikey, TTL y parciales
- Lección 07 — explain y consultas cubiertas
- Lección 08 — Pipeline de agregación
- Lección 09 — WiredTiger — caché, compresión y journal
- Lección 10 — Antipatrones frecuentes

Módulo 3: Replicación y sharding

- Lección 11 — Replica sets, oplog y elecciones
- Lección 12 — Write concern, read concern y read preference
- Lección 13 — Sharding — arquitectura y cuándo usarlo
- Lección 14 — Elegir la shard key
- Lección 15 — Change streams

Módulo 4: Seguridad, backup y producción

- Lección 16 — Backups — mongodump, instantáneas y clusters con sharding
- Lección 17 — Autenticación y control de acceso basado en roles
- Lección 18 — TLS, cifrado en reposo y cifrado de campos
- Lección 19 — Monitoreo — profiler, currentOp, oplog y caché
- Lección 20 — Proyecto final: MongoDB listo para producción

Módulo 1: Fundamentos y modelado de datos

Lección 01 — Qué es MongoDB y su modelo de documentos

Es una base de datos de documentos: guarda documentos BSON (similares a JSON, con más tipos) agrupados en colecciones, cada uno con un campo `_id` único, y los documentos de una misma colección pueden tener estructuras distintas.

Se pueden añadir campos nuevos sin migrar toda la colección y los documentos antiguos conviven con los nuevos; el riesgo es que sin reglas la colección acumule documentos inconsistentes, por lo que conviene validar la estructura.

Lección 02 — Modelado — incrustar o referenciar

Se incrusta (embedding) lo que se lee junto y tiene tamaño acotado, ganando lecturas en una sola operación y actualizaciones atómicas; se referencia cuando los datos crecen sin límite, se comparten entre muchos documentos o superan el límite de 16 MB por documento.

El documento puede alcanzar el límite de 16 MB y fallar la escritura, y los documentos muy grandes consumen memoria y cuestan más de actualizar; conviene referenciar o dividir en otra colección.

Lección 03 — Validación de esquema con \$jsonSchema

Se define un validador con `$jsonSchema` en la colección (campos obligatorios, tipos, rangos); `validationAction` decide si rechaza (error) o solo avisa (warn), y `validationLevel` si se aplica a todos los documentos o solo a los nuevos y a los modificados que antes cumplían (moderate).

Permite detectar documentos que incumplen sin romper las escrituras de la aplicación; se corrigen los datos o el código antes de pasar a rechazar, lo que reduce el riesgo de interrumpir el servicio.

Lección 04 — Operadores de actualización y atomicidad por documento

Modifican campos concretos de forma atómica en el servidor sin reescribir el documento entero desde la aplicación; con `upsert` se inserta si no existe, y una actualización de un solo documento es atómica incluso si toca campos anidados o arrays.

La operación se ejecuta de forma atómica en el servidor sin que otro cliente intercale cambios; leer y escribir por separado permite que dos clientes pisen el mismo valor, es decir, una condición de carrera.

Lección 05 — Transacciones multi-documento

Las operaciones sobre un único documento son atómicas, y por eso un buen modelado reduce la necesidad de transacciones; las transacciones multi-documento (replica sets desde 4.0, clusters con sharding desde 4.2) existen pero añaden coste y tienen por defecto un límite de duración de 60 segundos, así que deben ser breves.

Si los datos relacionados viven en un mismo documento, una sola escritura es atómica; las transacciones añaden coste y complejidad, e incrustar lo que cambia junto evita coordinar varios documentos.

Módulo 2: Índices y rendimiento

Lección 06 — Índices — compuestos (regla ESR), multikey, TTL y parciales

Un índice compuesto sigue la regla ESR: primero los campos de igualdad (Equality), luego los de ordenación (Sort) y por último los de rango (Range); los índices sobre arrays son multikey (una entrada por elemento), los TTL eliminan documentos por antigüedad y los parciales indexan solo los documentos que cumplen un filtro.

El índice se ordena por el primer campo y después por los siguientes, y solo se puede recorrer en ese orden; poner primero los campos de igualdad reduce el rango a recorrer, y un orden incorrecto obliga a examinar más entradas o a ordenar en memoria.

Lección 07 — explain y consultas cubiertas

Muestra si la consulta usa IXSCAN o COLLSCAN y compara documentos examinados con devueltos (cuanto más cerca de 1, mejor); una consulta cubierta se resuelve solo con el índice (0 documentos examinados) si la proyección usa solo campos indexados y excluye `_id` cuando este no está en el índice.

La consulta lee muchos documentos que luego descarta, desperdiciando lectura y memoria; suele faltar un índice adecuado o el índice no es selectivo, y un índice mejor acerca los examinados a los devueltos.

Lección 08 — Pipeline de agregación

Procesa documentos por etapas (`$match`, `$group`, `$sort`, `$lookup`...); conviene poner `$match` y `$project` al principio para reducir datos y poder usar índices, las etapas bloqueantes tienen límite de memoria (100 MB) y pueden necesitar `allowDiskUse` según la versión, y `$lookup` necesita un índice en el campo de la colección unida.

Reduce cuántos documentos pasan a las etapas siguientes y puede usar un índice, así las etapas posteriores trabajan con menos datos; un `$match` al final procesaría todo antes de filtrar.

Lección 09 — WiredTiger — caché, compresión y journal

Usa control de concurrencia a nivel de documento, comprime los datos (snappy por defecto) y tiene una caché interna que por defecto es la mayor entre el 50 % de (RAM – 1 GB) y 256 MB; el journal (registro de escritura previa) protege entre checkpoints, que se hacen aproximadamente cada 60 segundos.

El sistema operativo también necesita memoria para su caché de archivos y para el resto de procesos, y WiredTiger se apoya en esa caché para los datos comprimidos; sin margen se produce paginación y baja el rendimiento.

Lección 10 — Antipatrones frecuentes

Arrays sin límite y documentos enormes, índices de más (cada uno encarece las escrituras), expresiones regulares sin ancla al inicio donde se espera usar índice y paginar con skip de offsets grandes en lugar de usar una consulta por rango sobre un campo indexado (por ejemplo `_id` mayor que el último visto).

El servidor tiene que recorrer y descartar todos los documentos anteriores al offset, así que el coste crece con el número de documentos saltados; una consulta por rango sobre un índice empieza directamente en el punto correcto.

Módulo 3: Replicación y sharding

Lección 11 — Replica sets, oplog y elecciones

Tiene un primario que acepta escrituras y secundarios que replican leyendo su oplog; si el primario falla, los miembros con voto eligen uno nuevo por mayoría, por lo que se usan 3 miembros con datos (o un número impar de votos) y los drivers pueden reintentar escrituras durante la elección.

Para elegir primario hace falta mayoría de los votos y con dos miembros la mayoría es dos; al caer uno, el otro no la alcanza y se queda como secundario, por eso se usan tres miembros o un número impar de votos.

Lección 12 — Write concern, read concern y read preference

Con `w:1` solo confirma el primario, y si cae antes de replicar la escritura puede deshacerse (rollback) tras la elección; con `w:majority` se confirma cuando la mayoría la tiene y no se pierde en un failover; el `read concern majority` devuelve solo datos confirmados por la mayoría, y leer de secundarios puede devolver datos desactualizados.

Solo la había confirmado el primario, y si cae antes de replicarla los demás miembros no la tienen; el nuevo primario no contiene esa escritura y en el antiguo se deshace al volver, mientras que `w:majority` exige que la mayoría la tenga antes de confirmar.

Lección 13 — Sharding — arquitectura y cuándo usarlo

Reparte una colección entre varios shards (cada uno un replica set) según la shard key; los clientes se conectan a mongos, que enruta las consultas, y los config servers guardan los metadatos del cluster; se usa cuando los datos o la carga superan lo que soporta un solo replica set.

El sharding añade complejidad operativa y una mala shard key es difícil de corregir; muchos problemas de rendimiento se resuelven con mejores índices o modelo, y se shardea cuando los datos o la carga realmente superan un solo servidor.

Lección 14 — Elegir la shard key

Alta cardinalidad, baja frecuencia de valores y no ser monótona: una clave creciente como un timestamp u ObjectId envía todas las inserciones al último chunk (hot shard); una clave hash reparte mejor las escrituras pero hace que las consultas por rango visiten todos los shards (scatter-gather), mientras que incluir la shard key en la consulta permite dirigirla a un solo shard.

Los nuevos documentos tienen siempre valores mayores y caen en el último chunk, que está en un único shard; el resto de shards recibe pocas escrituras y queda infrautilizado, mientras que una clave hash distribuye los valores entre los chunks.

Lección 15 — Change streams

Permiten reaccionar a cambios (inserciones, actualizaciones, borrados) de una colección, base de datos o despliegue; se basan en el oplog, requieren un replica set o un cluster con sharding, y un resume token permite continuar tras una desconexión mientras el cambio siga dentro de la ventana del oplog.

El oplog es un registro circular que descarta las entradas más antiguas al llenarse; si el cambio ya salió de la ventana, el token deja de ser válido, y una ventana mayor da más margen para reanudar.

Módulo 4: Seguridad, backup y producción

Lección 16 — Backups — mongodump, instantáneas y clusters con sharding

mongodump/mongorestore hacen copias lógicas útiles para bases pequeñas (con --oplog se obtiene una copia consistente en un replica set), pero leen todos los datos y no escalan bien; en bases grandes se usan instantáneas del sistema de archivos o backup continuo con recuperación a un punto en el tiempo, y en clusters con sharding hay que coordinar la copia de todos los shards y de los config servers.

Los datos y los metadatos viven en componentes distintos y deben corresponder al mismo momento; copias tomadas en instantes distintos darían un cluster inconsistente, y el balanceador moviendo chunks complica la consistencia.

Lección 17 — Autenticación y control de acceso basado en roles

Hay que habilitar la autorización (security.authorization) y crear usuarios con roles de mínimo privilegio (read, readWrite, dbAdmin...) en lugar de usar roles de administración para las aplicaciones; los miembros de un replica set o cluster se autentican entre sí con keyFile o certificados x.509, y la instancia no debe escuchar en interfaces públicas.

Cualquiera que alcance el puerto puede leer, modificar o borrar todos los datos sin credenciales, y se han producido muchos secuestros de bases expuestas pidiendo rescate; la protección es habilitar la autorización y limitar la exposición de red.

Lección 18 — TLS, cifrado en reposo y cifrado de campos

TLS cifra el tráfico entre clientes y servidores y entre miembros del cluster; el cifrado en reposo nativo es una función de las ediciones Enterprise y Atlas, mientras que en Community se recurre a cifrado de disco o de sistema de archivos; para proteger campos sensibles incluso frente a un administrador de la base existe el cifrado a nivel de campo en el cliente (CSFLE).

Los datos se cifran antes de salir de la aplicación y el servidor solo ve texto cifrado; quien administre la base no puede leer los campos sin las claves, que se gestionan fuera de la base de datos.

Lección 19 — Monitoreo — profiler, currentOp, oplog y caché

El retraso de replicación y la ventana del oplog, las conexiones, la presión sobre la caché de WiredTiger (datos sucios y expulsiones) y las operaciones lentas: el profiler registra las que superan slowms (100 ms por defecto) y db.currentOp muestra las que están en ejecución; mongostat o un exporter a Prometheus ayudan a seguirlo.

Si un secundario se retrasa más que la ventana, ya no puede ponerse al día leyendo el oplog y tendría que hacer una resincronización completa, que es costosa; una ventana amplia da margen ante mantenimientos o fallos.

Lección 20 — Proyecto final: MongoDB listo para producción

Modelo con documentos acotados y esquema validado, índices diseñados con explain, replica set de 3 miembros con w:majority donde importe la durabilidad, sharding solo si hace falta con una shard key bien elegida, autorización con mínimo privilegio y TLS, backups probados con recuperación a un punto en el tiempo y monitoreo de retraso, oplog y operaciones lentas.

Intenta mencionar al menos dos pilares: modelo acotado con esquema validado, índices diseñados con explain, replica set de 3 miembros con w:majority, autorización con mínimo privilegio y TLS, o backups probados con monitoreo.